→ Attention:-

multi head attention:-

Q, K, V are all the input X.

- so we take X → (batch_len, seq_len, d_model)) and project them into a different dimension.
- a linear model will be used to project them.
  - model (d_model, d_model)
  - out_q = model(Q)
  - then transform out_q (batch_len, seq_len, dmodel) → q_proj (batch_len, seq_len, num_heads, d_modell)
- Transform from q_proj (batch_len, seq_len, num_heads, d_model) to q_proj (batch_len, num_heads, seq_len, d_model)
- then proform scaled dot product attention

$$\frac{Q \cdot K^T}{\sqrt{d_k}}$$ → then apply mask (essentially fill the indexes in mask with a '$-\infty$')
↓
apply softmax
↓
@V → result.

→ the result from scaled dot product attention is of shape (batch_size, num_heads, seq_len, d_model)
⇓ transpose
batch_size, seq_len, num_heads, d_model)
⇓
apply out put model (d_model × d_model)
⇓
reshape output to (batch_len, −1, d_model)

→ Positional encodings:-

$$PE = \begin{cases} \sin\left(pos/1000^{2i/d\_model}\right) \\ \cos\left(pos/1000^{2i/d\_model}\right) \end{cases}$$

create a pe matrix where terms must be

div_term will produce

$$\left[\frac{1}{1000^{0/D}}, \frac{1}{1000^{1/D}} \cdots\cdots \frac{1}{1000^{2i/D}}\right]$$

positions will be

$$\begin{bmatrix} 0 \\ \vdots \\ L-1 \end{bmatrix}$$

d → d_model
L → seq_len

pe = torch.sin/cos(positions * div)

$$PE = \begin{bmatrix} \sin(0) & \cos(0) & \sin(0) & \cdots \\ \sin(1) & \vdots & \vdots & \vdots \\ \sin(2) & \vdots & \vdots & \vdots \\ \vdots & & & \\ \sin(n-1) & & & \end{bmatrix}$$

div_terms are made using the formula

$$\exp\left(2i \cdot \frac{-\log(1000)}{D}\right) \approx 1/1000^{2i/d\_model}$$

→ Layernorm:-

$$\frac{x - mean}{\sqrt{var + eps}} * \gamma + \beta$$

(γ, β → trainable parameters)

Layer norm will only applied on the −1 dimension, which is each layer.

→ Batch Normalization:-

$$\frac{x - mean}{\sqrt{var + eps}} * \gamma + \beta$$

applied on each batch on 0 dimension.

→ Transformer:-

attention(x, x, x, mask) → dropout
↓ (add)
layer norm
↓
feed_forward
↓
dropout
↓ (add)
Layernorm

dropout is done before 2 layer norms.

→ Language Model:-

embeddings → positional-encodings → create a mask (for attention)
↓
transformers (can be a list)
→ linear layer (to flatten output dims)

* before passing inputs to the language model, we need to convert the inputs into tokens.

→ Tokenization:-

normal tokenization:

- convert the text by breaking down words to characters including all the symbols. (data sample level)
- now store all the unique characters.
- sort these unique chars to form a vocabulary. and len of the vocabulary will be the vocab size.
- map the indices of characters in the form below
  {ch: i} → characters in the vocabulary.
- now make index using the {ch: i} mapping,
- store id's of the characters present in each sample by mapping them to the stoi.
- make pairs:- suppose we have ids for chars like c, h, a, r
  0, 1, 2, 3
  it creates pairs like {0, (1, 2, 3)}
  {(0, 1) (2, 3)} {(0, 1, 2), (3)}
- now initialize a seq_len
  use a sliding window approach to converts the data into chunks

  all_ids = all input ids from every sample.
  so each chunk will have
  x = all_ids[i : i+seq_len]
  y = all_ids[i+1 : i+seq_len+1]

  input_ids will be x and labels will be y.
- this is all for training set,
  for testing set, we don't want to create a stoi, instead use the stoi from training.
  vocab_size will automatically be the len of stoi).

→ BPE tokenizer:-

Byte pair encoding tokenizer

First given the text convert the chalacters into bytes(Hex) and then to list ↓
list(text.encode('utf-8'))
each char takes 1 bytes and some complex chars like emoji's take up 4 bytes.

- maintain a dict token_to_char stores integer keys and bytes val for 65th index value will be b'A'.

- we will have to specify a vocab size. so BPE only uses 256 ids. (↳ hyper-parameter.)
- if hyper-parameter is 300, then 44 merges must happen.
- create pairs and generate a counter for the pairs.
- get the most freq pair and now use a next_id
  store the next_id in token_to_char with value of concatenation of the pairs.
- create a variable which tracks the merges.
- increment next_id +1.

→ encoding using BPE:
essentially comes down to merging pairs stored in merges.

merging pairs →
create a new token list where, you check
single iteration {
for the occurence of the pair, if there is one
instead of sending in pair(0) and pair(1) into list, we send in the next id (both are (int))
}

for each merge pair in merges, run this iteration.

→ decode:-
for token in tokens:
append the utf decoded value of the token using utf.decode and the bytes stored in token_to_char.
if token is not recognizable, add empty char to the text.

→ Training Process:-

- scaler → torch.amp.GradScaler
  ↓
  apply it on loss, optimizer.

  torch.amp.autocast
  ↓
  use it for calculating logits
- Grad accumulation steps:-
  scale loss for number of Grad accumulation steps.

→ Metrics:-

Perplexity = math.exp(avg_loss)
↓ how confident model is while making predictions.

→ generate text:-

* first generate logits, and scale by temperature
  logits[:, −1, :] / temperature.
* instead of using argmax to obtain highest probs. use multinomial sampling.
* next_token_id = torch.multinomial(probs, num_samples=1)
* concatenate the next_token_id with x.

run this iteration for max_new_token times.

at last use tokenizer.decode to convert into text.

→ difference between multinomial sampling vs argmax?

	probs
be →	0.7
banana →	0.1
to →	0.2

argmax picks be since high prob score

but multinomial sampling picks randomly b/w three, like a dice rolling with three probable outcomes.